

SIEVE OF ERATOSTHENES

Use when: many primality checks/prime lists needed up to N.

```
const int MAXN = 1000005;
vector<bool> isComposite(MAXN, false); vector<int> primes;
void sieve() { // O(N log log N)
    for (int i=2; i<MAXN; i++) {
        if (!isComposite[i]) {
            primes.push_back(i);
            for (long long j=(long long)i*i; j<MAXN; j+=i) isComposite[j]=true;
        }
    }
}
bool isPrimeFast(int x) { return x >= 2 && !isComposite[x]; } // O(1) after sieve
```

PRIME FACTORIZATION

Use when: factorizing many numbers – precompute spf once.

```
vector<int> spf(MAXN);
void sieveSPF() { // smallest prime factor sieve
    for (int i=2; i<MAXN; i++) if (spf[i]==0)
        for (int j=i; j<MAXN; j+=i) if (spf[j]==0) spf[j]=i;
}
vector<int> factorize(int x) { // O(log n) using SPF
    vector<int> f; while (x > 1) { f.push_back(spf[x]); x /= spf[x]; }
    return f;
}
vector<pair<int,int>> factorizeBasic(int n) { // O(sqrt n), {prime, exponent}
    vector<pair<int,int>> res;
    for (int p=2; (long long)p*p<=n; p++) {
        if (n % p != 0) continue;
        int cnt=0; while (n % p == 0) { n /= p; cnt++; }
        res.push_back({p, cnt});
    }
    if (n > 1) res.push_back({n, 1});
    return res;
}
```

DIVISORS & MULTIPLES COUNTING

```
int countDivisors(int n) { // O(sqrt n)
    int cnt=0;
    for (int i=1; (long long)i*i<=n; i++) if (n%i==0) { cnt++; if (i!=n/i) cnt++; }
    return cnt;
}
vector<int> getDivisors(int n) { // O(sqrt n)
    vector<int> d;
    for (int i=1; (long long)i*i<=n; i++) if (n%i==0) { d.push_back(i); if (i!=n/i)
d.push_back(n/i); }
    return d;
}
```

Use when: divisor counts needed for every number up to N at once – $\sim O(N \log N)$.

```
vector<int> divCount(MAXN, 0); // O(N log N) precompute
for (int i=1; i<MAXN; i++) for (int j=i; j<MAXN; j+=i) divCount[j]++;

// #divisors =  $\prod(\text{exponent} + 1)$ 
```